1-5 · Docker Compose 이해하고 사용하기 · 정리·회고 0.5H

1주차 총정리 · 산출물 제출 확인

손으로 하던 일을 파일로 옮겨 적어 두면 재현된다

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 1주차 다섯 차시가 남긴 결과물을 한 표로 되짚는다

② Compose 명령 여덟 개를 요약 표로 정리한다

③ 자주 겪는 오류 셋의 원인과 해결을 확인한다

④ 환경을 정돈하고 산출물 제출을 확인한다

2주차부터는 이 위에 진짜 데이터를 얹습니다

1주차에 지나온 길

스크립트 → Dockerfile → Compose, 다섯 차시가 남긴 것

차시	배운 것	남은 결과물
1-1	리눅스 명령과 파이프	access.log · hello.sh
1-2	쉘 스크립트와 종료 코드	backup.sh 등 스크립트 5종
1-3	이미지·컨테이너·볼륨·포트	pgdata 볼륨 · 도커 명령 감각
1-4	Dockerfile 과 레이어 캐시	app/ 폴더 · data-app 이미지
1-5	Compose 로 환경 전체 정의	yml · README.md (산출물)

한 주를 관통한 생각은 하나 — 손으로 하던 일을 파일로 옮겨 적어 두면 언제든 똑같이 재현된다

1주차가 이어지는 곳

이번 주에 익힌 것이 2~8주차에서 다시 나온다

1주차	이어지는 곳
1-1 grep · tail · 파이프	4 · 5 · 6 · 8주차 — 컨테이너 로그를 걸러 읽는다
1-2 종료 코드 · cron	5-5 마트 스크립트 → 8주차 Airflow 가 재시도 판단
1-3 run · -p · exec · 볼륨	2주차 MySQL · MongoDB 기동 · 4-1 minio-data
1-4 이미지에 비밀값 금지	4-2 · 7-1 — API 키는 .env 로
1-5 Compose · 서비스 이름	4 MinIO · 5 Spark · 6 Kafka · 8 Airflow

healthcheck 와 Dockerfile 직접 빌드는 이후 주차에 다시 나오지 않는다 — 심화 과제로 익혀 둔다

Compose 명령 요약

전부 docker compose 뒤에 붙인다

하려는 일	명령
문법 검사	config
기동 (빌드 포함)	up -d --build
상태 확인	ps -a
로그 보기	logs 서비스명 (-f 실시간)
안으로 들어가기	exec 서비스명 bash (없으면 sh)
한 서비스만 재시작	restart 서비스명
정지·삭제 (볼륨 유지)	down
볼륨까지 삭제	down -v (되돌릴 수 없음)

docker-compose.yml 이 있는 폴더에서 — down -v 는 되돌릴 수 없으니 평소에는 down 만

2주차를 위한 정돈

~/data-eng/week1 에서 네 줄

1. **docker compose down**
   이번 주 환경 내리기 — 볼륨은 남겨 둔다
2. **docker rm pg 2>/dev/null**
   3차시 pg 컨테이너가 남아 있으면 정리
3. **docker ps -a · docker images**
   남은 것 확인 — 실행 중인 컨테이너가 없어야 한다
4. **mkdir -p ~/data-eng/week2**
   2주차 폴더 미리 만들어 두기

> **확인 포인트: 실행 중인 컨테이너가 없고 ~/data-eng 아래에 week1 · week2 폴더가 있으면 정돈 완료**

오늘의 체크리스트

여덟 개 중 몇 개에 표시할 수 있나요

1. YAML 들여쓰기 규칙(공백 2칸 · 탭 금지)을 설명할 수 있다
2. services · volumes · depends_on 이 각각 무엇을 정의하는지 안다
3. 컨테이너 안 localhost 와 서비스 이름의 차이를 실험으로 확인했다
4. up -d --build 로 db·admin·app 3개 서비스를 한 번에 기동했다
5. pgAdmin 에서 호스트 이름 db 로 PostgreSQL 에 연결했다
6. down 후 up 을 해도 표가 남는 것으로 볼륨 보존을 확인했다
7. down -v 후 README 의 명령만으로 재현되는지 스스로 검증했다
8. 주차 산출물(docker-compose.yml + README.md)을 제출했다

자주 겪는 오류

증상 → 원인 → 해결

증상	원인	해결
yaml: line N 파싱 오류	들여쓰기 어긋남 · 탭 섞임	공백 2칸 통일 · config 로 검사
port is already allocated	3차시 pg 가 5432 사용 중	docker stop pg 뒤 up -d
depends_on 인데 DB 연결 실패	시작 순서만 보장, 준비 완료 아님	잠시 후 재시도 · healthcheck

세 번째의 근본 해결은 healthcheck + depends_on 의 condition — 앱 쪽 재시도 로직도 실무 표준

예상 질문

자주 나오는 네 가지

1. **docker-compose 와 docker compose 는 다른가요**
   하이픈은 옛 파이썬 버전, 공백은 도커 내장 현재 버전 — 이 과정은 전부 공백

2. **파일 이름이 꼭 docker-compose.yml 이어야 하나요**
   기본값일 뿐 — 다른 이름은 docker compose -f 파일명 으로 지정

3. **컨테이너가 재시작을 반복해요**
   docker compose logs 서비스명 — 대부분 환경변수 누락이나 설정 오류

4. **2주차부터는 무엇을 하나요**
   MySQL · MongoDB — 1-3 의 docker run · exec 로 띄운다. Compose 는 4주차 MinIO 부터 다시 쓴다

다음 회차 예고

2-1 · MySQL 9.7 설치 및 기초 쿼리 (4H)

01

이론 1H

관계형 DB · MySQL 9.7

표 · 행 · 열 · 기본키

02

실습 1H

Docker 로 MySQL 기동

클라이언트 접속

03

실습 1.5H

DDL · DML · SELECT

WHERE · GROUP BY

04

정리 0.5H

리뷰 · Q&A

shop DB 는 2주 내내

준비: Docker 실행(Windows · macOS 는 Desktop, 리눅스는 systemctl) — docker version 이 버전을 출력하면 준비 완료

1-5 · 정리·회고 0.5H 마무리

1주차 마무리 — 다음은 2-1 MySQL 9.7

손으로 하던 일을 파일로 옮겨 재현하는 법을 익혔습니다. 2주차부터는 그 위에 진짜 데이터를 얹습니다

- ✓ 환경 정돈: 실행 중 컨테이너 없음 · week2 폴더
- ✓ 산출물 두 파일 제출 확인
- ✓ 심화 과제(선택): healthcheck · .env · 바인드 마운트 · Git
- ✓ 2-1 준비: Docker 실행 상태(운영체제별)

다음 회차

2-1 MySQL 9.7 설치 및 기초 쿼리